

Alright 🚀 let's set up a **Kubernetes cluster on Ubuntu 20.04** using **kubeadm** (most common for labs, training, and even prod foundations).

We'll build:

- 1 Control Plane (Master)
  - 1+ Worker Nodes
- 

### 🧰 Prerequisites (ALL NODES)

Run on **every node** (master + workers).

#### 1 Set hostname (example)

```
sudo hostnamectl set-hostname master-node # on master
```

```
sudo hostnamectl set-hostname worker-node1 # on worker
```

Add entries in /etc/hosts (all nodes):

```
sudo nano /etc/hosts
```

```
192.168.1.10 master-node
```

```
192.168.1.11 worker-node1
```

---

#### 2 Disable Swap (Kubernetes requirement)

```
sudo swapoff -a
```

```
sudo sed -i 's/^swap /s/^#/' /etc/fstab
```

---

#### 3 Enable Required Kernel Modules

```
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
```

```
overlay
```

```
br_netfilter
```

```
EOF
```

```
sudo modprobe overlay
```

```
sudo modprobe br_netfilter
```

---

### Set Sysctl Params

```
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
```

```
net.bridge.bridge-nf-call-iptables = 1
```

```
net.bridge.bridge-nf-call-ip6tables = 1
```

```
net.ipv4.ip_forward = 1
```

```
EOF
```

```
sudo sysctl --system
```

---

### Install Container Runtime (containerd)

```
sudo apt update
```

```
sudo apt install -y containerd
```

Generate default config:

```
sudo mkdir -p /etc/containerd
```

```
containerd config default | sudo tee /etc/containerd/config.toml
```

Enable Systemd cgroup driver:

```
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml
```

Restart:

```
sudo systemctl restart containerd
```

```
sudo systemctl enable containerd
```

---

### Install Kubernetes Components

```
sudo apt update
```

```
sudo apt install -y apt-transport-https ca-certificates curl
```

Add Kubernetes repo:

```
curl -fsSL https://packages.cloud.google.com/apt/doc/apt-key.gpg | sudo apt-key add -
```

```
cat <<EOF | sudo tee /etc/apt/sources.list.d/kubernetes.list
```

```
deb https://apt.kubernetes.io/ kubernetes-xenial main
```

```
EOF
```

Install packages:

```
sudo apt update
```

```
sudo apt install -y kubelet kubeadm kubectl
```

```
sudo apt-mark hold kubelet kubeadm kubectl
```

---

### Initialize Control Plane (MASTER NODE ONLY)

```
sudo kubeadm init --pod-network-cidr=192.168.0.0/16
```

When it finishes, you'll see a **kubeadm join command** — save it!

---

### Configure kubectl for your user

```
mkdir -p $HOME/.kube
```

```
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
```

```
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

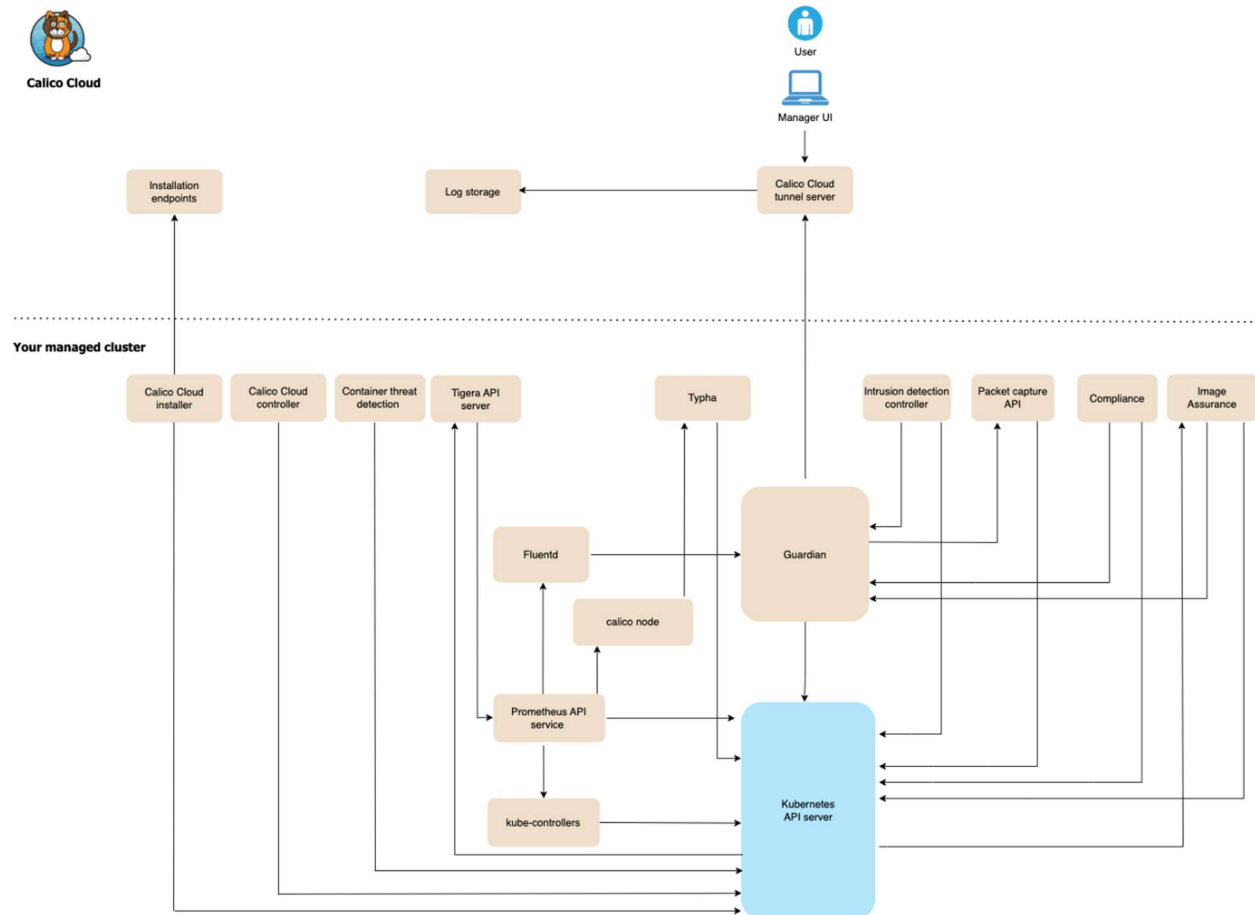
Test:

```
kubectl get nodes
```

Node will be **NotReady** (expected, no network yet).

---

### Install Pod Network (Calico)



kubectl apply -f <https://docs.projectcalico.org/manifests/calico.yaml>

Wait 1–2 minutes:

kubectl get nodes

Master should become **Ready** ✅

## + Join Worker Nodes

Run on **each worker** using the command from kubeadm init:

```
sudo kubeadm join 192.168.1.10:6443 \
```

```
--token <token> \
```

```
--discovery-token-ca-cert-hash sha256:<hash>
```

---

### **Verify Cluster**

On master:

```
kubectl get nodes
```

```
kubectl get pods -A
```

You should see:

- control plane node
  - worker node(s)
  - calico pods running
- 

### **Test Deployment**

```
kubectl create deployment nginx --image=nginx
```

```
kubectl expose deployment nginx --type=NodePort --port=80
```

```
kubectl get svc
```

Access:

```
http://<worker-node-ip>:<nodeport>
```

---

### **(Optional) Allow Master to Run Pods (Lab Only)**

```
kubectl taint nodes --all node-role.kubernetes.io/control-plane-
```

---

### **Your Cluster is Ready!**

You now have:

- ✓ kubeadm-based cluster
- ✓ containerd runtime
- ✓ Calico networking
- ✓ Multi-node ready setup

